

Brief — Tests unitaires en Java

Prouver que son code fonctionne, et le garder sous contrôle quand il évolue

En bref

Jour 2 : vous n'écrivez plus de nouveaux algorithmes, vous **prouvez que ceux d'hier fonctionnent**. Vous testerez les exercices que vous avez réalisés, chacun part de sa propre production. Le fil rouge de la journée : un test n'est pas là pour faire joli, il est là pour **vous prévenir quand quelque chose casse**.

1. Compétences travaillées

Compétence	Ce que vous démontrez
C9. Préparer et exécuter les plans de tests d'une application (niveau 1)	Écrire des tests unitaires qui vérifient une méthode, cas nominaux et cas limites.
C3. Développer des composants métier (niveau 1)	Faire évoluer une méthode existante sans casser ce qui marchait.

2. Mise en place de JUnit

Changement par rapport à hier : on travaille aujourd'hui **dans IntelliJ**, plus en ligne de commande. Raison : JUnit n'est pas fourni avec Java, c'est une bibliothèque externe : l'IDE se charge de la récupérer et de lancer les tests pour vous.

1. Créez une classe de test à côté de la classe à tester, nommée sur le même modèle : `Calculator` donne `CalculatorTest`.
2. Écrivez une première méthode annotée `@Test`. L'annotation apparaît en rouge : placez le curseur dessus et faites `Alt + Entrée`, puis choisissez « Add JUnit5 to classpath ». IntelliJ télécharge JUnit tout seul.
3. Lancez le test avec la flèche verte dans la marge, à gauche du nom de la méthode (ou clic droit puis Run).

Astuce : placez vos classes de test dans le même dossier que vos exercices. Sinon, vos méthodes devront être déclarées `public` pour être visibles depuis le test.

3. Anatomie d'un test

Astuces à retenir

- **La méthodologie** : Construisez vos test en reprenant la logique GIVEN (mes donnée de départ), WHEN (la méthode que j'appelle) THEN (le résultat que j'attends)
- **L'ordre compte** : `assertEquals(attendu, obtenu)`. Inversé, le message d'erreur vous racontera l'inverse de la réalité.
- **Un test = un cas**. Mieux vaut cinq petits tests bien nommés qu'un gros test qui vérifie tout.
- **Nommez pour être compris** : `addition_twoPositifsNumbers()` est plus parlant que `test1()`.
- **Nombres à virgule** : ajoutez une tolérance, sinon les arrondis vous piègent : `assertEquals(2.5, division(5, 2), 0.0001)`.
- **Booléens** : `assertTrue(...)` et `assertFalse(...)`.
- **Vérifiez que vos tests savent échouer** : cassez volontairement votre méthode et relancez. Si le test reste vert, c'est qu'il ne teste rien.

4. Les exercices à tester

Vous testez **les exercices que vous avez faits hier**. Le socle attendu : exercices 1 à 4. Les suivants sont des bonus.

Exercice 1 — Calculatrice (ensemble, en classe)

On écrit le premier test ensemble (celui de l'addition, ci-dessus). Vous enchaînez ensuite seuls sur **soustraction**, **multiplication**, **division** et **sommeDeAaB** : le modèle est le même à chaque fois.

Astuces : pour la division, n'oubliez pas la tolérance. Pour **sommeDeAaB**, pensez au cas où *a* et *b* sont égaux (la somme de 4 à 4 vaut 4).

Exercice 2 — FizzBuzz, puis la règle qui change tout

Cet exercice se fait en cinq temps. Suivez-les dans l'ordre, l'intérêt est dans le déroulement.

1. Écrivez un test pour chacune de ces valeurs : 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 30. Tout doit être vert.
2. Le client change d'avis : il veut une règle de plus. **Tout nombre divisible par 2 affiche « Qix »**. Modifiez votre méthode pour l'ajouter.
3. Relancez vos tests **sans y toucher**. Observez ce qui se passe.
4. Une question se pose : 6 est divisible par 3 *et* par 2. Que doit-il renvoyer ? On retient la convention de la concaténation : 6 donne **FizzQix**, 30 donne **FizzBuzzQix**. Déduisez-en les autres.
5. Corrigez le code *et* les tests pour revenir au vert..

Ce qu'il faut retenir : vos tests vous ont indiqué *exactement* quelles valeurs la nouvelle règle impactait. Sans eux, il aurait fallu tout revérifier à la main.

Exercice 3 — Manipulation de chaînes (libre)

Testez **compterOccurrences** et **estPalindrome** comme bon vous semble. **Astuce** : les cas limites sont les plus instructifs : une lettre absente du texte, un mot d'une seule lettre, un mot vide.

Exercice 4 — Mot de passe : du booléen à la liste d'erreurs

Un *true* ou *false* ne dit pas à l'utilisateur *ce qui ne va pas*. Faites évoluer votre méthode pour qu'elle renvoie **toutes les erreurs rencontrées** :

Nouvelle signature

```
static List<String> verifierMotDePasse(String mdp)
```

```
// Liste vide = le mot de passe est valide
```

```
// Liste pleine = un message par règle non respectée
```

Fixez vos messages une bonne fois pour toutes (vos tests s'appuieront dessus), par exemple « Au moins 12 caractères », « Au moins un chiffre », etc. **Important** : la méthode ne doit plus s'arrêter à la première erreur, elle doit toutes les collecter.

Vos assertions portent alors sur deux choses :

- le nombre d'erreurs : `assertEquals(3, erreurs.size());`
- leur contenu : `assertTrue(erreurs.contains("Au moins un chiffre"))`.

Écrivez suffisamment de tests : un mot de passe valide (liste vide), un cas par règle violée, et surtout **un mot de passe qui viole plusieurs règles à la fois** — c'est là qu'on voit l'intérêt de la liste par rapport au booléen. Par exemple, "abc" cumule trois manquements.

Exercices 6 et 7 — Romains et Potter (exploration libre)

Si vous les avez faits, testez-les à votre façon. Aucune consigne imposée : c'est l'occasion d'expérimenter par vous-même.

5. Modalités pédagogiques

Paramètre	Détail
Durée	1 jour.
Outil	IntelliJ IDEA (JUnit 5). Pas de ligne de commande aujourd'hui.
Organisation	Premier test écrit ensemble, puis chacun avance sur ses propres exercices.

6. Livrables

Sur votre dépôt GitHub privé partagé avec les formateurs :

Livable	Attendu	Fait ?
Classes de tests	Une classe de test par exercice testé, tous les tests au vert.	☐
FizzBuzz avec la règle Qix	Code et tests mis à jour, cohérents entre eux.	☐
Mot de passe	Méthode renvoyant la liste des erreurs, avec ses tests.	☐

7. Critères de performance

Checklist d'auto-positionnement. Objectif : pouvoir cocher chaque item honnêtement.

- **Mise en place** : j'ai créé une classe de test et je sais lancer mes tests dans IntelliJ.
 - **Écriture** : mes tests portent un nom qui dit ce qu'ils vérifient, et chacun teste un seul cas.
 - **Cas limites** : je ne teste pas que les cas évidents, je pense aussi aux cas particuliers.
 - **Régression** : j'ai vu mes tests passer au rouge après un changement, et j'ai su quoi corriger.
 - **Liste d'erreurs** : je sais vérifier la taille d'une liste et son contenu dans un test.
-